

Multichannel HF SDR Receiver with Icecast Streaming

Build Manual — Raspberry Pi 4 + SDRplay RSP2 Pro

Example Station

v2.0 — 2026

1. Overview
 - 1.1 Target frequencies (example configuration)
 - 1.2 Architecture summary
 - 1.3 Design principles followed throughout
2. Hardware and OS Installation
 - 2.1 Components
 - 2.2 Base OS
 - 2.3 Initial hardening checklist
3. Remote Access (Cloudflare Tunnel)
4. Dedicated Service Account and Directory Layout
5. SDRplay API and SDRangel (built from source)
 - 5.1 SDRplay API
 - 5.2 Why SDRangel must be built from source
 - 5.3 Build dependencies
 - 5.4 Build
6. SDRangel Configuration
 - 6.1 systemd unit
 - 6.2 REST API device and channel provisioning
 - 6.3 Verifying reception
 - 6.4 ALSA loopback audio routing
7. MP3 Encoding and Icecast Streaming
 - 7.1 Install ffmpeg
 - 7.2 Per-channel credentials file (kept separate from the unit file)
 - 7.3 Per-channel streaming script
 - 7.4 systemd service — indefinite automatic recovery
 - 7.5 Verifying resilience
8. Simultaneous Multi-Reader Access (Streaming + Scheduled Recording)
9. Firewall Rules Summary
10. SIP Readiness (prepared, not activated)
 - 10.1 Install
 - 10.2 Per-instance configuration
 - 10.3 Outbound-only registration and its firewall implications
 - 10.4 Systemd services (per instance, not shown in full)
11. Operational Reference
 - 11.1 File and directory layout
 - 11.2 systemd units
 - 11.3 Key verification commands
 - 11.4 Known startup timing

1. Overview

This manual documents the build of a headless, unattended, 24/7 multichannel HF receiver station. A single SDRplay RSP2 Pro, driven by a Raspberry Pi 4, captures one wideband IQ slice of HF spectrum and derives an arbitrary number of independent demodulator channels from it, in any mix of SSB (LSB/USB) and AM. Each channel's audio is encoded to MP3 and streamed continuously to an Icecast server, with automatic recovery from network and server interruptions.

This build has an automated companion. Everything in this manual — including support for any number of channels and mixed LSB/USB/AM modes — is also available as `hamhfrx-installer`, a set of idempotent, resumable shell scripts with an accompanying install guide PDF, bundled alongside this manual. The manual remains the authoritative reference for what each step does and why; the installer is that same knowledge automated. Building by hand from this manual is still fully supported and is the better choice if you want to understand or modify each step as you go.

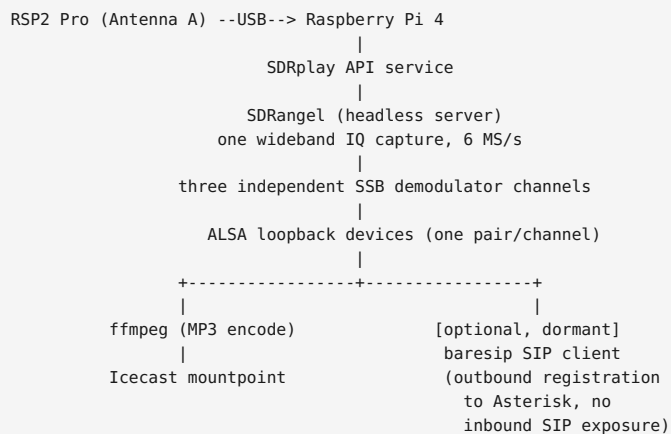
The design also provisions (but does not activate) an outbound-only SIP signaling path, so the same audio channels can later be made available as dialable extensions on an existing Asterisk PBX with no inbound ports ever opened on the receiver's firewall.

1.1 Target frequencies (example configuration)

The receiver is not limited to three channels or to SSB — this is simply the configuration used as the worked example throughout this manual. Section 6.2 covers the general case, including AM.

Channel	Frequency	Mode	Notes
1	3600 kHz	LSB	80 m
2	7100 kHz	LSB	40 m
3	4700 kHz	USB	utility

1.2 Architecture summary



1.3 Design principles followed throughout

- **Everything outside the user's home directory.** All scripts, configs, and the SDRangel install live under `/opt/`, owned by a dedicated, unprivileged, no-login system service account (`hamsvc`).
- **Everything survives a cold reboot with zero manual steps.** Every component is a systemd unit with correct `After=` / `Requires=` ordering.
- **Remote-access-first.** The station is administered exclusively through an outbound Cloudflare Tunnel; no inbound ports are opened for administration, and (in the SIP-ready configuration) none are opened for telephony either.
- **Verify empirically, not by assumption.** Every configuration value used in this manual (ALSA device names, REST API endpoints, antenna port numbering, etc.) was confirmed against the live system's own reported state rather than assumed from documentation, since exact strings and behavior can vary by software version.

2. Hardware and OS Installation

2.1 Components

- Raspberry Pi 4, 8 GB RAM
- SDRplay RSP2 Pro
- Good-quality, short (<1 m) USB cable, connected to a Pi 4 USB3 (blue) port
- Official Raspberry Pi power supply

2.2 Base OS

Install **Raspberry Pi OS (Legacy) Lite, Bookworm (Debian 12), 64-bit**.

Two other options were evaluated and rejected:

- **Standard (non-Legacy) Raspberry Pi OS based on Debian 13 (Trixie)** — rejected because, at time of writing, building SDRangel from source on Trixie/aarch64 hits an unresolved upstream linker error involving `libunwind`. Debian 13-based images may become viable once this is resolved upstream.
- **Desktop variants** — unnecessary overhead for a headless appliance; Lite is the correct choice.

Flash with Raspberry Pi Imager, setting hostname, user account, and SSH public key at flash time via the imager's advanced options where possible.

2.3 Initial hardening checklist

```
sudo hostnamectl set-hostname sdr-station-1
# edit /etc/hosts: 127.0.1.1 <-> new hostname

sudo apt update && sudo apt full-upgrade -y
sudo reboot
```

SSH — key-only authentication

```
# on the workstation: generate a key, copy the public key to the Pi's
# ~/.ssh/authorized_keys (chmod 700 ~/.ssh, chmod 600 authorized_keys)

sudo nano /etc/ssh/sshd_config
```

Set:

```
PasswordAuthentication no
PermitRootLogin no
AllowUsers <youruser>
```

```
sudo systemctl restart ssh
```

Firewall (default-deny inbound)

```
sudo apt install -y ufw
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw enable
```

fail2ban (note: Raspberry Pi OS Lite has no `rsyslog` by default, so fail2ban's `sshd` jail must read the systemd journal directly)

```
sudo apt install -y fail2ban
sudo nano /etc/fail2ban/jail.local
```

```
[sshd]
enabled = true
port = ssh
backend = systemd
maxretry = 4
bantime = 3600
findtime = 600
```

```
sudo systemctl enable --now fail2ban
```

Unattended security-only upgrades

```
sudo apt install -y unattended-upgrades apt-listchanges
sudo dpkg-reconfigure -plow unattended-upgrades
```

In `/etc/apt/apt.conf.d/50unattended-upgrades`, leave only the `Debian-Security`-labeled origin lines active; comment out the general `label=Debian` line so routine (non-security) package churn — a real risk to a USB/audio-dependent appliance — never applies automatically. Leave `Automatic-Reboot` set to `false`.

Time, disabled radios, logging

```
sudo timedatectl set-timezone <Region/City>
sudo timedatectl set-ntp true
sudo rfkill block bluetooth
sudo rfkill block wifi # if running on Ethernet
```

Reduce SD card wear

```
sudo dphys-swapfile swapoff
sudo systemctl disable dphys-swapfile
sudo apt purge -y dphys-swapfile
```

Add `noatime` to the root and boot mount options in `/etc/fstab`. Consider `log2ram` or forwarding logs to a remote syslog target, and use a high-endurance SD card or boot from a USB SSD for genuinely unattended long-term service.

Hardware watchdog

```
sudo apt install -y watchdog
```

In `/etc/watchdog.conf`: `watchdog-device = /dev/watchdog`. Ensure `dtparam=watchdog=on` is present in `/boot/firmware/config.txt`.

```
sudo systemctl enable --now watchdog
```

3. Remote Access (Cloudflare Tunnel)

The station is administered exclusively through an outbound Cloudflare Tunnel — no inbound port is ever opened for SSH.

```
curl -L https://github.com/cloudflare/cloudflared/releases/latest/download/cloudflared-linux-arm64.deb -o cloudflared.deb
sudo dpkg -i cloudflared.deb
cloudflared tunnel login
cloudflared tunnel create sdr-station-1
```

Create `/etc/cloudflared/config.yml` :

```
tunnel: <tunnel-UUID>
credentials-file: /etc/cloudflared/<tunnel-UUID>.json

ingress:
- hostname: <flat-single-level-subdomain>.<yourdomain>
  service: ssh://localhost:22
- service: http_status:404
```

Certificate note: the free Cloudflare Universal SSL certificate only covers the apex domain and one level of subdomain. Use a flat hostname such as `sdr-station-1-ssh.example.com` , not a nested one — a nested hostname requires the paid Advanced Certificate Manager add-on.

```
cp ~/.cloudflared/<tunnel-UUID>.json /etc/cloudflared/
cloudflared tunnel route dns sdr-station-1 sdr-station-1-ssh.example.com
sudo cloudflared service install
sudo systemctl enable --now cloudflared
```

In the Cloudflare Zero Trust dashboard, create an **Access application** protecting that hostname (e.g. email one-time-PIN policy). For day-to-day SSH client use (PuTTY, etc.), install `cloudflared` on the workstation too and configure a **Local** proxy type pointing at:

```
cloudflared access ssh --hostname sdr-station-1-ssh.example.com
```

This authenticates once via browser (cached per the Access session duration) and thereafter connects silently, keeping the private key local to the workstation rather than pasted into any browser session.

Once verified working end-to-end, remove the direct SSH allowance:

```
sudo ufw delete allow OpenSSH
```

4. Dedicated Service Account and Directory Layout

All application components run under a dedicated, unprivileged, no-login system account, and all files live under `/opt/`.

```
sudo useradd --system --home-dir /opt/sdr-station-1 --create-home \
--shell /usr/sbin/nologin hamsvc

sudo mkdir -p /opt/sdr-station-1/bin /opt/sdr-station-1/streaming

sudo usermod -aG plugdev hamsvc  # USB/SDRplay device access
sudo usermod -aG audio hamsvc   # ALSA device access (critical – see 6.4)

sudo chown -R hamsvc:hamsvc /opt/sdr-station-1
```

5. SDRplay API and SDRangel (built from source)

5.1 SDRplay API

Download the current **ARM64/aarch64 Linux** API installer directly from <https://www.sdrplay.com/downloads/> (always fetch current, do not reuse a cached version number).

```
chmod +x SDRplay_RSP_API-Linux-*.run
sudo ./SDRplay_RSP_API-Linux-*.run
sudo systemctl enable --now sdrplay
```

This installs `libsdrplay_api.so` under `/usr/local/lib`, headers under `/usr/local/include`, and a background `sdrplay_apiService` daemon managed by its own systemd unit. Verify:

```
lsusb | grep -i sdrplay
sudo systemctl status sdrplay
```

5.2 Why SDRangel must be built from source

Prebuilt community Docker images for `sdrangelsrv` on arm64 are commonly built **without** the SDRplay API present at compile time. This silently omits the `sdrplayv3` plugin (which provides RSP2/RSPduo/RSPdx-specific features: antenna port selection, external reference clock, etc.) and leaves only a legacy RSP1-only plugin. There is no runtime fix for this — the plugin must be present at compile time. Building from source, on Bookworm, with the API already installed on the host, is the reliable path.

5.3 Build dependencies

```
sudo apt install -y build-essential cmake git pkg-config \
  qtbase5-dev qtchooser qt5-qmake qtbase5-dev-tools \
  qtmultimedia5-dev libqt5multimedia5-plugins \
  qttools5-dev qttools5-dev-tools libqt5svg5-dev \
  qtdeclarative5-dev qtpositioning5-dev qtlocation5-dev \
  libqt5serialport5-dev libqt5websockets5-dev libqt5opengl5-dev \
  libfftw3-dev libusb-1.0-0-dev libasound2-dev zlib1g-dev libxml2-dev \
  libqt5charts5-dev libqt5texttospeech5-dev qtwebengine5-dev \
  libboost-all-dev libopus-dev
```

These were determined by iterating `cmake` configure runs to completion; a different SDRangel version may require a slightly different set. If configure reports a missing required package (a hard `CMake Error`, as opposed to a “not found, feature disabled” warning), install the corresponding `-dev` package and re-run configure.

5.4 Build

```
cd ~
git clone https://github.com/f4exb/sdrangel.git
cd sdrangel
git checkout v7.22.7 # or the current stable tag
export CXXFLAGS='-O3 -mtune=cortex-a72'
export CFLAGS='-O3 -mtune=cortex-a72'
mkdir build && cd build
cmake -Wno-dev -DCMAKE_BUILD_TYPE=Release -DBUILD_GUI=OFF -DBUILD_SERVER=ON \
  -DCMAKE_INSTALL_PREFIX=/opt/install/sdrangel ..
```

Confirm in the configure output:

```
-- SDRPLAY_LIBRARIES /usr/local/lib/libsdrplay_api.so
-- Found SDRPLAY: ...
```

and confirm no unresolved `CMake Error` remains (missing *optional* hardware plugins — BladeRF, HackRF, LimeSDR, RTL-SDR, Airspy, UHD, etc. — reporting “not found” is expected and correct; those plugins are simply skipped).

Build inside `tmux` (this takes 45–90+ minutes on a Pi 4 and must survive an SSH/tunnel disconnect):

```
sudo apt install -y tmux
tmux new -s sdrbuild
make -j$(nproc)
```

Verify the target plugin exists before installing:

```
find ~/sdrangel/build -iname "*sdrplayv3*"
# expect: .../lib/pluginssrv/libinputsdrplayv3srv.so
```

Install and hand ownership to the service account:

```
sudo make install
sudo chown -R hamsvc:hamsvc /opt/install/sdrangel
```

6. SDRangel Configuration

6.1 systemd unit

```
sudo mkdir -p /etc/systemd/system/sdrangelsrv.service.d
sudo nano /etc/systemd/system/sdrangelsrv.service
```

```
[Unit]
Description=SDRangel headless server
After=network.target sdrplay.service
Wants=sdrplay.service

[Service]
Type=simple
User=hamsvc
ExecStart=/opt/install/sdrangel/bin/sdrangelsrv
Restart=on-failure
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Real-time audio scheduling (required — see 6.4) and **module-load ordering** (required — see 6.5) are added via a drop-in override:

```
sudo nano /etc/systemd/system/sdrangelsrv.service.d/override.conf
```

```
[Unit]
After=network.target sdrplay.service systemd-modules-load.service
Wants=sdrplay.service
Requires=systemd-modules-load.service

[Service]
CPUSchedulingPolicy=rr
CPUSchedulingPriority=50
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now sdrangelsrv
```

6.2 REST API device and channel provisioning

`sdrangelsrv` exposes a REST API on port 8091 with **no persistent state across restarts for device sets/channels** — these must be recreated every time the process starts. This is handled by a provisioning script, run as a one-shot systemd unit chained after the server.

Key facts learned empirically about the API, worth preserving:

- Adding a device set requires an explicit `POST /sdrangel/deviceset` call before anything else will succeed.
- `POST .../channel` **ignores** any settings embedded in the same request body in this SDRangel version — a channel must be created bare, then configured with a separate `PATCH .../channel/{n}/settings` call.
- Changing `audioDeviceName` via `PATCH` on an already-open channel does **not** force the audio stream to reopen against the new device in this version; a fresh channel creation (or a full device-set teardown and rebuild) is required for a device-name change to take effect.
- Presets are saved via `POST /sdrangel/preset` (create) and updated via `PUT /sdrangel/preset` (update an *existing* preset only — it will error if the named preset does not already exist).

```
sudo -u hamsvc nano /opt/sdr-station-1/bin/provision-sdrangel.sh
```

```
#!/bin/bash
set -e

API="http://127.0.0.1:8091/sdrangel"

timeout 60 bash -c "until curl -s -o /dev/null '$API/presets'; do \
    echo 'Waiting for sdrangelsrv API...'; sleep 1; done"

curl -s -X POST "$API/deviceset" > /dev/null
curl -s -X PUT "$API/deviceset/0/device" -H "Content-Type: application/json" \
    -d '{"hwType": "SDRplayV3", "sequence": 0}' > /dev/null
curl -s -X PATCH "$API/deviceset/0/device/settings" -H "Content-Type: application/json" \
    -d '{"deviceHwType": "SDRplayV3", "sdrPlayV3Settings": {"centerFrequency": 5350000, "devSampleRate": 6000000,
"bandwidthIndex": 7, "log2Decim": 0}}' > /dev/null
curl -s -X POST "$API/deviceset/0/device/run" > /dev/null

sleep 20 # allow RSP2 Pro firmware upload and RF settle to complete

# Channel 0: 3600 kHz LSB -> Loopback card 0
curl -s -X POST "$API/deviceset/0/channel" -H "Content-Type: application/json" \
    -d '{"channelType": "SSBDemod", "direction": 0}' > /dev/null
curl -s -X PATCH "$API/deviceset/0/channel/0/settings" -H "Content-Type: application/json" \
    -d '{"channelType": "SSBDemod", "SSBDemodSettings": {"inputFrequencyOffset": -1750000, "rfBandwidth": -3000, "lowCutoff":
-300, "audioDeviceName": "hw:CARD=Loopback,DEV=0", "audioMute": 0, "title": "3600 kHz LSB"}}' > /dev/null

# Channel 1: 7100 kHz LSB -> Loopback 1
curl -s -X POST "$API/deviceset/0/channel" -H "Content-Type: application/json" \
    -d '{"channelType": "SSBDemod", "direction": 0}' > /dev/null
curl -s -X PATCH "$API/deviceset/0/channel/1/settings" -H "Content-Type: application/json" \
    -d '{"channelType": "SSBDemod", "SSBDemodSettings": {"inputFrequencyOffset": 1750000, "rfBandwidth": -3000, "lowCutoff":
-300, "audioDeviceName": "hw:CARD=Loopback_1,DEV=0", "audioMute": 0, "title": "7100 kHz LSB"}}' > /dev/null

# Channel 2: 4700 kHz USB -> Loopback 2
curl -s -X POST "$API/deviceset/0/channel" -H "Content-Type: application/json" \
    -d '{"channelType": "SSBDemod", "direction": 0}' > /dev/null
curl -s -X PATCH "$API/deviceset/0/channel/2/settings" -H "Content-Type: application/json" \
    -d '{"channelType": "SSBDemod", "SSBDemodSettings": {"inputFrequencyOffset": -650000, "rfBandwidth": 3000, "lowCutoff":
"audioDeviceName": "hw:CARD=Loopback_2,DEV=0", "audioMute": 0, "title": "4700 kHz USB"}}' > /dev/null

echo "Provisioning complete."
```

```
sudo chmod +x /opt/sdr-station-1/bin/provision-sdrangel.sh
```

Frequency-offset and sideband convention: `inputFrequencyOffset` is the target frequency minus the device center frequency, in Hz. For LSB, both `rfBandwidth` and `lowCutoff` must be **negative**; for USB, both stay **positive**. This mirrors the filter passband around zero Hz — it is not two independent signs.

6.2.1 Generalizing beyond three fixed SSB channels

The three-channel LSB/LSB/USB example above is a worked example, not a ceiling. Two things generalize cleanly:

Any number of channels. Repeat the create-then-PATCH pair once per channel, incrementing the channel index (`.../channel/0/settings` , `.../channel/1/settings` , ...) and the loopback card (`Loopback` , `Loopback_1` , `Loopback_2` , `Loopback_3` , ...). The only real constraint is that every channel's frequency must fall within the usable span of the device's sample rate around its center frequency — recompute the center as the midpoint between your lowest and highest channel, and widen the sample rate if the spread is large relative to it (roughly 80% of sample rate as clean usable bandwidth is a reasonable rule of thumb).

AM, alongside LSB/USB. AM has no sideband, so it uses a different SDRangel channel type (`AMDemod`) and a single **symmetric** bandwidth value rather than the signed `rfBandwidth` / `lowCutoff` pair SSB uses:

```
# AM channel example — 6000 kHz, offset computed the same way as any
# other channel (target frequency minus device center frequency)
curl -s -X POST "$API/deviceset/0/channel" -H "Content-Type: application/json" \
    -d '{"channelType": "AMDemod", "direction": 0}' > /dev/null
curl -s -X PATCH "$API/deviceset/0/channel/3/settings" -H "Content-Type: application/json" \
    -d '{"channelType": "AMDemod", "AMDemodSettings": {"inputFrequencyOffset": 750000, "rfBandwidth": 6000, "audioDeviceName": "hw:CARD=Loopback_3,DEV=0", "audioMute": 0, "title": "6100 kHz AM"}}' > /dev/null
```

As with every other value in this manual, these AM field names (`AMDemodSettings` , `rfBandwidth`) were current for SDRangel v7.22.7 at time of writing but were not independently verified against a physical AM signal the way the SSB convention was.

Confirm with `curl -s http://127.0.0.1:8091/sdrangel/deviceset/0/channel/<n>/settings` against your own running instance before trusting it blindly — the same “verify empirically” principle this whole manual follows.

The automated installer’s `channels.conf` (see the companion install guide) implements exactly this generalization — arbitrary channel count, per-channel LSB/USB/AM selection, automatic center-frequency and span calculation — if you’d rather not hand-maintain the provisioning script as the channel list grows.

```
sudo nano /etc/systemd/system/sdrangel-provision.service
```

```
[Unit]
Description=Provision SDRangel device set and channels
After=sdrangelsrv.service
Requires=sdrangelsrv.service

[Service]
Type=oneshot
User=hamsvc
ExecStart=/opt/sdr-station-1/bin/provision-sdrangel.sh

[Install]
WantedBy=multi-user.target
```

```
sudo systemctl daemon-reload
sudo systemctl enable --now sdrangel-provision
```

6.3 Verifying reception

```
curl -s http://127.0.0.1:8091/sdrangel/deviceset/0/channel/0/report | python3 -m json.tool
```

Returns `channelPowerDB` and `squelch` — a real, non-floor reading confirms the antenna, RSP2 Pro, and demod chain are working end to end.

6.4 ALSA loopback audio routing

Each demodulator channel’s audio is routed to a dedicated ALSA loopback device rather than any real hardware output.

```
echo "options snd-aloop enable=1,1,1 index=10,11,12" | sudo tee /etc/modprobe.d/snd-aloop.conf
echo "snd-aloop" | sudo tee -a /etc/modules
sudo modprobe snd-aloop enable=1,1,1 index=10,11,12
aplay -l | grep -i loopback
```

Each loopback **card** is a cross-connected pair: what is written to `DEV=0` on a card is what appears on `DEV=1` of the *same* card, and vice versa. SDRangel writes to `DEV=0`; downstream consumers (ffmpeg, baresip) read from `DEV=1` of the same card.

Two permission requirements are easy to miss and will silently produce a fully “successful”-looking but empty audio pipeline if skipped:

1. The service account must be a member of the `audio` Linux group, or `sdrangelsrv`’s Qt-based audio backend will enumerate **zero** real ALSA devices (falling back to a non-functional placeholder) with no error indicating why.
2. The `snd-aloop` kernel module must be loaded **before** `sdrangelsrv` starts, or the same empty-enumeration symptom occurs — this is what the `Requires=systemd-modules-load.service` / `After=...systemd-modules-load.service` lines in section 6.1 exist to guarantee.

Always verify the exact device name string SDRangel itself reports (`GET /sdrangel/audio`) rather than assuming a `hw:10,0` -style hand-typed name — the correct form is `hw:CARD=Loopback,DEV=0` .

7. MP3 Encoding and Icecast Streaming

Each channel is captured from its loopback's `DEV=1` side and pushed to a distinct Icecast mountpoint by an independent `ffmpeg` process, managed by its own `systemd` service for indefinite automatic recovery.

7.1 Install ffmpeg

```
sudo apt install -y ffmpeg
ffmpeg -codecs 2>/dev/null | grep -i mp3 # confirm libmp3lame present
```

7.2 Per-channel credentials file (kept separate from the unit file)

```
sudo -u hamsvc nano /opt/sdr-station-1/streaming/icecast-ch1.env
```

```
ICECAST_URL=icecast://source:<password>@<icecast-host>:8000/<mountpoint>
```

```
sudo chmod 600 /opt/sdr-station-1/streaming/icecast-ch1.env
sudo chown hamsvc:hamsvc /opt/sdr-station-1/streaming/icecast-ch1.env
```

Repeat for `icecast-ch2.env` and `icecast-ch3.env` with their own mountpoints/credentials.

7.3 Per-channel streaming script

Using a real script file (rather than an inline `bash -c` string in the unit file) avoids `systemd`'s unit-file escape-sequence parsing entirely, which otherwise mishandles the `sed` backreferences used below.

```
sudo -u hamsvc nano /opt/sdr-station-1/streaming/stream-ch1.sh
```

```
#!/bin/bash
set -e
source /opt/sdr-station-1/streaming/icecast-ch1.env

exec stdbuf -oL -eL /usr/bin/ffmpeg -hide_banner -loglevel warning \
  -f alsa -i hw:CARD=Loopback,DEV=1 \
  -af volume=10dB -ac 1 -channel_layout mono -ar 44100 \
  -codec:a libmp3lame -b:a 64k -content_type audio/mpeg \
  -f mp3 "$ICECAST_URL" 2>&1 | sed -u -E 's#(source:)[^@]*(@)#\1REDACTED\2#'
```

```
sudo chmod +x /opt/sdr-station-1/streaming/stream-ch1.sh
```

Notes:

- `-ac 1 -channel_layout mono` is required together — `-ac 1` alone does not guarantee the LAME encoder emits a true single-channel stream, and at 64 kbit/s the difference is audible (mono gives the full bitrate budget to one real channel of content).
- The trailing `sed` filter redacts the Icecast source password from every line of `ffmpeg`'s own output before it reaches the `systemd` journal — necessary because `ffmpeg` prints the full connection URL, credentials included, into its own error/status messages.
- The `-af volume=10dB` gain boost was applied only to the channel found to have the weakest received signal during testing; apply per-channel gain as needed based on each channel's actual `channelPowerDB`, not uniformly.

Repeat for `stream-ch2.sh` (source `icecast-ch2.env`, capture `hw:CARD=Loopback_1,DEV=1`) and `stream-ch3.sh` (source `icecast-ch3.env`, capture `hw:CARD=Loopback_2,DEV=1`).

7.4 systemd service — indefinite automatic recovery

```
sudo nano /etc/systemd/system/stream-ch1.service
```

```
[Unit]
Description=MP3 Icecast stream - 3600 kHz LSB
After=network-online.target sdrangelsrv.service
Wants=network-online.target
Requires=sdrangelsrv.service
StartLimitIntervalSec=0

[Service]
Type=simple
User=hamsvc
ExecStart=/opt/sdr-station-1/streaming/stream-ch1.sh
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

StartLimitIntervalSec=0 is the single most important line for a 24/7 unattended stream. Systemd's default behavior is to give up permanently after 5 restarts within 10 seconds. A sustained outage (Icecast server down for an extended period, ISP outage, etc.) would exhaust that default limit almost immediately and leave the stream permanently dead until someone manually restarts it. Setting the interval to `0` disables the ceiling entirely: `ffmpeg` will retry every `RestartSec` indefinitely, and reconnect automatically the moment the destination becomes reachable again, with no manual intervention.

Repeat for `stream-ch2.service` and `stream-ch3.service`, each pointing at its own script.

```
sudo systemctl daemon-reload
sudo systemctl enable --now stream-ch1 stream-ch2 stream-ch3
```

7.5 Verifying resilience

Confirm the rate limit is genuinely disabled:

```
systemctl show stream-ch1 -p StartLimitIntervalUSec -p StartLimitBurst -p NRestarts
```

Expect `StartLimitIntervalUSec=0`.

To prove recovery from a genuine severed connection (not just a process restart with the network never having actually dropped — a firewall rule alone will **not** interrupt an already-established TCP connection):

```
sudo pkill -9 ffmpeg
sleep 10
sudo systemctl status stream-ch1 --no-pager
```

The service should show a fresh restart and a clean reconnection in its journal within one `RestartSec` cycle.

8. Simultaneous Multi-Reader Access (Streaming + Scheduled Recording)

A raw ALSA `hw:` device permits only one reader at a time. If a scheduled local recording needs to run concurrently with the live stream on the same channel, both consumers must instead read from the `dsnoop` variant of the loopback capture device (e.g. `dsnoop:CARD=Loopback,DEV=1`), which allows multiple simultaneous readers of the same source. Apply this device name to both the streaming script and any recording script for a given channel.

9. Firewall Rules Summary

```
sudo ufw status numbered
```

Maintain only the rules actually in active use. Example baseline for the configuration in this manual (streaming-only, SIP not yet activated):

```
8091/tcp    ALLOW IN    <admin LAN subnet>/24    # SDRangel REST API / Swagger UI
8080/tcp    ALLOW IN    <admin LAN subnet>/24    # (as applicable)
```

Everything else (SSH) is reached exclusively via the Cloudflare Tunnel, not a direct ufw allow rule.

10. SIP Readiness (prepared, not activated)

The design supports adding outbound-only SIP telephony later with no inbound ports ever opened, by having each channel's loopback `DEV=1` side also captured by a **baresip** instance that registers outward to an existing Asterisk server.

10.1 Install

```
sudo apt install -y baresip-core
dpkg -L baresip-core | grep -i alsa # confirm alsa.so is present
```

10.2 Per-instance configuration

Each of the three SDR channels needs its own baresip instance with unique local ports (three instances cannot share one config directory):

Instance	SIP listen	Audio listen	HTTP listen	Web TCP listen	Capture device
ch1	0.0.0.0:5060	127.0.0.1:5561	127.0.0.1:8001	127.0.0.1:4441	hw: CARD=Loopback,DEV=1
ch2	0.0.0.0:5070	127.0.0.1:5562	127.0.0.1:8002	127.0.0.1:4442	hw: CARD=Loopback_1,DEV=1
ch3	0.0.0.0:5080	127.0.0.1:5563	127.0.0.1:8003	127.0.0.1:4443	hw: CARD=Loopback_2,DEV=1

In each instance's `config` file, at minimum:

```
answermode          auto
```

(required — there is no one present to manually answer an inbound call) and bind all of

`cons_listen / http_listen / ctrl_tcp_listen` to `127.0.0.1` only, never `0.0.0.0` — these are local control interfaces, not intended to be reachable from the network.

10.3 Outbound-only registration and its firewall implications

Because there is no TLS in this deployment (plain UDP registration to Asterisk on port 5060), the “outbound only” property is enforced at the **network layer**, not the protocol layer: the SIP listen socket that baresip opens is a real, responsive UDP listener. What makes it safe behind a home/office router with no port forwarding is NAT statefulness — only return traffic on the connection baresip itself initiated can reach it. If the device's public-facing exposure ever changes (a real public IP, a DMZ setting, etc.), that protection disappears and the listener becomes directly reachable from the internet.

Mitigation used in this design: restrict the SIP and RTP ports to accept traffic only from the known Asterisk server IP, regardless of NAT state, as defense in depth:

```
sudo ufw allow from <asterisk-ip> to any port 5060 proto udp
sudo ufw allow from <asterisk-ip> to any port 5070 proto udp
sudo ufw allow from <asterisk-ip> to any port 5080 proto udp
sudo ufw deny 5060/udp
sudo ufw deny 5070/udp
sudo ufw deny 5080/udp

# RTP media ports (narrow, non-overlapping ranges, one per instance)
sudo ufw allow from <asterisk-ip> to any port 10000:10099 proto udp
sudo ufw allow from <asterisk-ip> to any port 10100:10199 proto udp
sudo ufw allow from <asterisk-ip> to any port 10200:10299 proto udp
```

Rule order matters: the `allow from <asterisk-ip>` rules must be added **before** the corresponding blanket `deny` rules, since ufw/netfilter evaluates rules in order and stops at the first match.

Set matching `rtp_ports` ranges (e.g. `10000-10099`) in each baresip instance's `config` file so the actual negotiated media ports fall inside the firewalled range.

10.4 Systemd services (per instance, not shown in full)

Each baresip instance should run as its own systemd service, `User=hamsvc`, `ExecStart=/usr/bin/baresip -f /opt/sdr-station-1/baresip-ch<N>`, with `Restart=always` and appropriate `After=/ Requires=sdrangelsrv.service` ordering, following the same pattern established for the streaming services in section 7.4. Accounts (the actual Asterisk credentials) are configured in

each instance's `accounts` file once available.

11. Operational Reference

11.1 File and directory layout

```
/opt/sdr-station-1/          service account home
├─ bin/
│   └─ provision-sdrangel.sh
├─ streaming/
│   ├── icecast-ch1.env  icecast-ch2.env  icecast-ch3.env  (mode 600)
│   └─ stream-ch1.sh    stream-ch2.sh    stream-ch3.sh
├─ baresip-ch1/  baresip-ch2/  baresip-ch3/  (config, accounts, contacts)
└─ /opt/install/sdrangel/          SDRangel build output
```

11.2 systemd units

sdrplay.service	SDRplay API hardware service
sdrangelsrv.service	SDRangel headless server (+ RT scheduling and module-load ordering overrides)
sdrangel-provision.service	one-shot: device set + 3 channels
cloudflared.service	remote admin access
stream-ch1/2/3.service	MP3 encode + push to Icecast
[baresip-ch1/2/3.service]	dormant until SIP is activated

11.3 Key verification commands

```
# full status
sudo systemctl status sdrplay sdrangelsrv sdrangel-provision \
  cloudflared stream-ch1 stream-ch2 stream-ch3 --no-pager

# USB stability (look for a single clean firmware-load transition,
# not a repeating disconnect/reconnect cycle)
sudo journalctl -k -b --no-pager | grep -i usb | tail -50

# power
vcgencmd get_throttled      # expect 0x0

# per-channel signal / demod status
curl -s http://127.0.0.1:8091/sdrangel/deviceset/0/channel/<N>/report | python3 -m json.tool

# ALSA loopback devices present
aplay -l | grep -i loopback

# confirm restart-rate limiting is disabled on a streaming service
systemctl show stream-ch1 -p StartLimitIntervalUSec -p NRestarts
```

11.4 Known startup timing

A full cold boot to fully-provisioned, streaming state takes approximately 2–3 minutes, dominated by the RSP2 Pro's mandatory firmware upload (which occurs on every session open, not just first boot) and the provisioning script's deliberate settle delay. This is normal, not a fault.